Santhosh S

Software Fresher Evaluation's report

Submitted on Jul 12 2025 15:33:20 IST

Powered by doselect

265.0 (96.4%)
scored out of 275

Passed
in the assessment (Cut-off score >= 40%)

10
problems attempted out of 10

3.9 / 5
avg. code quality score

Negligible Violation
flagged by DoSelect proctoring engine

Test time analysis

51m 0s
time taken for completion

Jul 09 2025 20:06:55 IST
test invite time

Jul 12 2025 14:42:20 IST
test start time

Jul 12 2025 15:33:20 IST
test end time

Performance summary

9
solutions accepted

1
solution rejected

Proctor analysis

1
browser used

2
navigation violation

1
webcam violations

0.05 mins
test window violation

Plagiarism Not Detected
0 / 2 applicable questions

Webcam Violation - flagged by DoSelect Proctoring Engine due to below reasons

Total Frames Captured : 3058

Frames with Matching Faces	**3057**
Frames with Multiple Faces	**0**
Frames with Different Face	**1**
Frames with No Face	**0**

Total Frames Missing : 2

Webcam not detected	**0**
Test-taker closing the tab	**0**
Network Issues	**0**
Other factors*	**2**

Total Webcam Violations : 1

Set of 10 back-to-back Suspicious Frames**	**0**
Set of 10 back-to-back Missing frames	**0**
Suspicious Frames**/Missing Frames detected in less than 5% of test duration	**1**

* Missing frames due to other factors such as test-taker's system issues etc

** Suspicious frames includes Multiple Faces, Different Faces and No Face

Identity Image

Solutions

Problem Name	Problem Type	Status	Score
React: React Asynchronous Testing	MCQ	ACCEPTED	**5.0** / 5
React: Project	MCQ	ACCEPTED	**5.0** / 5
React: Behavior	MCQ	REJECTED	**0.0** / 10
In Node Js	MCQ	ACCEPTED	**20.0** / 20
LinkedList and Indexing Plagiarism Not Detected	Coding	ACCEPTED	**100.0** / 100
Ribbit the Frog Plagiarism Not Detected	Coding	ACCEPTED	**100.0** / 100
Which Oops Concept	MCQ	ACCEPTED	**5.0** / 5
OOPs: ATM System Minimum Balance Rule	MCQ	ACCEPTED	**20.0** / 20
RDBMS: Data Integrity	MCQ	ACCEPTED	**5.0** / 5
HTML DOM	MCQ	ACCEPTED	**5.0** / 5

Technology used

JavaScript (NodeJS)

Detailed Report

Powered by doselect

Problem 1 : React: React Asynchronous Testing

MCQ SCORE: **5**

Which statement is true regarding the waitFor function in asynchronous testing with React Testing Library?

Solution

ACCEPTED SCORE: **5.0** / 5

It throws an error if the condition is met immediately.

It is used to immediately get a DOM element without waiting.

It has a default timeout of 1000 milliseconds.

CANDIDATE'S ANSWER **CORRECT ANSWER**

It can be used to wait for the disappearance of UI elements.

Problem 2 : React: Project

MCQ | SCORE: **5**

In a React project that includes a component with a **large amount** of state data, which of the following techniques should you use to **optimize performance** and avoid unnecessary re-renders?

Solution

ACCEPTED | SCORE: **5.0** / 5

Use PureComponent to automatically optimize the component's rendering.

Use shouldComponentUpdate to prevent re-rendering of the component.

CANDIDATE'S ANSWER **CORRECT ANSWER**

Use React.memo to memoize the component's output and avoid unnecessary re-renders.

Use the useMemo hook to memoize the component's state data and avoid unnecessary re-renders.

Problem 3 : React: Behavior

MCQ | SCORE: **10**

You are building a **large-scale e-commerce** platform using React. You have multiple components that need to **share code**.

Which of the following React patterns would you use to achieve this functionality?

Solution

REJECTED | SCORE: **0.0** / 10

- React Context
- Higher-Order Components (HOCs)
- **CANDIDATE'S ANSWER** Redux
- **CORRECT ANSWER** React Render Props

Problem 4 : In Node Js

MCQ SCORE: **20**

Donald wants to write a code with **CRUD** operations without using a **framework** in node js. How would he implement the program?

Solution

ACCEPTED SCORE: **20.0** / 20

```
http.createServer(function(req, res) {
res.writeHead(200, {
'Content-Type': 'text/html'
});
if(url === '/about') {
res.write('<h1>about us page<h1>');
res.end();
} else if(url === '/contact') {
res.write('<h1>contact us page<h1>');
res.end();
} else {
res.write('<h1>Hello World!<h1>');
res.end();
}
}).listen(3000, function() {
console.log("server start at port 3000");
});
```

```
http.createServer(function(req, res) {
res.writeHead(200, {
'Content-Type': 'text/html'
});
res.end();
} else if(url === '/contact') {
res.write('<h1>contact us page<h1>');
res.end();
} else {
res.write('<h1>Hello World!<h1>');
res.end();
}
}).listen(3000, function() {
console.log("server start at port 3000");
});
```

```
var http = require('http'); //create a server object:
http.createServer(function(req, res) {
res.writeHead(200, {
'Content-Type': 'text/html'
});
if(url === '/about') {
res.write('<h1>about us page<h1>');
res.end();
} else if(url === '/contact') {
```

```
res.write('<h1>contact us page<h1>');
res.end();
} else {
res.write('<h1>Hello World!<h1>');
res.end();
}
})
```

CANDIDATE'S ANSWER

CORRECT ANSWER

```
var http = require('http');
http.createServer(function(req, res) {
res.writeHead(200, {
'Content-Type': 'text/html'
});
if(url === '/about') {
res.write('<h1>about us page<h1>');
res.end();
} else if(url === '/contact') {
res.write('<h1>contact us page<h1>');
res.end();
} else {
res.write('<h1>Hello World!<h1>');
res.end();
}
}).listen(3000, function() {
console.log("server start at port 3000");
});
```

Problem 5 : LinkedList and Indexing

CODING | SCORE: **100**

Problem Statement

You are given a linked list, you have to return the count of nodes in which **node value % node index = 0 or 1.** Node value is equal to the value, that node is storing. The node index is the position of that node from the beginning. (starting from 1).

Input Format

- First-line will contain an integer n, denoting the number of nodes in the LinkedList.
- The next n line will contain an integer each.

Constraints

- $1 \leq n \leq 5 * 10^5$
- $-10^4 \leq$ node-> value $\leq 10^4$

Output Format

- Return a single integer denoting the count.

Evaluation Parameters

- **Sample Input**

```
5
1
3
5
6
7
```

- **Sample Output**

```
2
```

- **Explanation**

Node with value 1 and node with value 3 only satisfy the condition.

1 % 1 == 0 and 3 % 2 == 1.

Solution

ACCEPTED | SCORE: **100.0** / 100

Code Quality Analysis

Minor quality violations
Quality score: 3.7

Plagiarism Not Detected
Max. Similarity: 0

JavaScript (NodeJS)

```javascript
'use strict';

process.stdin.resume();
process.stdin.setEncoding('utf-8');

let inputString = '';
let currentLine = 0;

process.stdin.on('data', function(inputStdin) {
        inputString += inputStdin;
});

process.stdin.on('end', function() {
        inputString = inputString.split('\n');

        main();
});

function readLine() {
        return inputString[currentLine++];
}



const SinglyLinkedListNode = class {
        constructor(nodeData) {
                this.data = nodeData;
                this.next = null;
        }
};

const SinglyLinkedList = class {
        constructor() {
                this.head = null;
                this.tail = null;
        }

        insertNode(nodeData) {
                const node = new SinglyLinkedListNode(nodeData);

                if (this.head == null) {
                        this.head = node;
                } else {
                        this.tail.next = node;
                }

                this.tail = node;
        }
};

function printSinglyLinkedList(node) {
        while (node != null) {
                console.log(String(node.data))

                        node = node.next;
        }
}


/*
```

Powered by doselect

```
 * Complete the 'getCount' function below.
 *
 * The function is expected to return an INTEGER.
 * The function accepts INTEGER SINGLY LINKED LIST head as parameter.
 *
 */

/*
 * For your reference:
 *
 * SinglyLinkedListNode {
 *      int data;
 *      SinglyLinkedListNode next;
 * }
 *
 */

function getCount(head) {
        let count =0;
        let index = 1;
        let current = head;
        while(current !== null){
                const mod = current.data % index;
                if (mod == 0 || mod == 1){
                        count++;
                }
                current =current.next;
                index++;
        }
        return count;
}

function main() {
        let head = new SinglyLinkedList();

        const headCount = parseInt(readLine().trim(), 10);

        for (let i = 0; i < headCount; i++) {
                const headItem = parseInt(readLine().trim(), 10);
                head.insertNode(headItem);
        }

        const result = getCount(head.head)

        console.log(result)
}
```

Evaluation Details

Testcase #7 *(weight:2)*

Status	Passed
Execution time	0.88s
CPU	0s
Memory	29MB
Description	Testcase passed! The solution's output matches the expected output.

Testcase #6 *(weight:3)*

Status	Passed

Execution time	0.98s
CPU	0s
Memory	68MB
Description	Testcase passed! The solution's output matches the expected output.

Testcase #2 *(weight:1)*

Status	Passed
Execution time	0.95s
CPU	0s
Memory	29MB
Description	Testcase passed! The solution's output matches the expected output.

Testcase #1 *(weight:1)*

Status	Passed
Execution time	0.86s
CPU	0s
Memory	29MB
Description	Testcase passed! The solution's output matches the expected output.

Testcase #5 *(weight:3)*

Status	Passed
Execution time	1.00s
CPU	0s
Memory	68MB
Description	Testcase passed! The solution's output matches the expected output.

Testcase #4 *(weight:1)*

Status	Passed
Execution time	0.87s
CPU	0s
Memory	28MB
Description	Testcase passed! The solution's output matches the expected output.

Testcase #0 *(sample)*

Status	Passed
Execution time	0.85s
CPU	0s
Memory	29MB
Description	Testcase passed! The solution's output matches the expected output.

Input

```
5
1
3
5
6
7
```

Expected output

```
2
```

Testcase #3 *(weight:2)*

Status	Passed
Execution time	0.87s
CPU	0s
Memory	28MB
Description	Testcase passed! The solution's output matches the expected output.

Testcase #8 *(weight:1)*

Status	Passed
Execution time	1.01s
CPU	0s
Memory	29MB

Description	Testcase passed! The solution's output matches the expected output.

Problem 6 : Ribbit the Frog

CODING SCORE: **100**

Problem Statement

You are given a non-empty Linked List **A**. The index of a node in A is decided by its distance from the head of the node(The head of the linked list is at 0-index and the node pointed by the next of the head is at 1 index and so on). All nodes have a value associated with them represent by **data**. Ribbit the frog starts from the head node and then jumps to the node in the linked list whose index is **(i+value of data in the node)%size of the linked list** and keeps on doing so from the next node that he has jumped on as per our indexing scheme. Ribbit doesn't like visiting familiar nodes which are the nodes that he has already visited, so he stops jumping as soon as he visits a node with an index that he has already visited. Return an ordered array whose elements sequentially contain the **value of data** of the nodes that he visits.

Input Format

- The first line contains **n** the number of nodes in the linked list **A**.
- The next **n** lines contain the elements of **A** sequentially from the head till the end of the linked list.

Constraints

- **1 <= n <= 10^3**
- **1 <= node->data <= 10^4**

Output Format

- Return an array as said in the problem statement.

Evaluation Parameters

- **Sample Input**

```
5
1
2
3
4
5
```

- **Sample Output**

```
1
2
4
3
```

- **Explanation**

Initially, he is on the head node with index 0 and that is counted as a visited node.

Next, he goes to the node with index 1(index of current node+data value of current node), then to node 4, and then to node 3, but after that, he will have to jump to the node with index 0 which has already been visited, so he will stop jumping at that point.

Solution

ACCEPTED | SCORE: **100.0** / 100

Code Quality Analysis

Maintainable code
Quality score: 4.1

Plagiarism Not Detected
Max. Similarity: 0

JavaScript (NodeJS)

```javascript
'use strict';

process.stdin.resume();
process.stdin.setEncoding('utf-8');

let inputString = '';
let currentLine = 0;

process.stdin.on('data', function(inputStdin) {
        inputString += inputStdin;
});

process.stdin.on('end', function() {
        inputString = inputString.split('\n');

        main();
});

function readLine() {
        return inputString[currentLine++];
}



const SinglyLinkedListNode = class {
        constructor(nodeData) {
                this.data = nodeData;
                this.next = null;
        }
};

const SinglyLinkedList = class {
        constructor() {
                this.head = null;
                this.tail = null;
        }

        insertNode(nodeData) {
                const node = new SinglyLinkedListNode(nodeData);

                if (this.head == null) {
                        this.head = node;
                } else {
                        this.tail.next = node;
                }

                this.tail = node;
        }
};

function printSinglyLinkedList(node) {
        while (node != null) {
```

```
                console.log(String(node.data))

                        node = node.next;
        }
}


/*
 * Complete the 'jumpedOn' function below.
 *
 * The function is expected to return an INTEGER ARRAY.
 * The function accepts INTEGER SINGLY LINKED LIST ll as parameter.
 *
 */

/*
 * For your reference:
 *
 * SinglyLinkedListNode {
 *      int data;
 *      SinglyLinkedListNode next;
 * }
 *
 */

function jumpedOn(head) {
        const values =[];
        let current = head;

        while(current){
                values.push(current.data);
                current = current.next;
        }

        const n = values.length;
        const visited = new Set();
        const result = [];
        let index = 0;

        while(!visited.has(index)){
                result.push(values[index]);
                visited.add(index);
                index = (index + values[index]) % n;
        }
        return result;
}

function main() {
        let ll = new SinglyLinkedList();

        const llCount = parseInt(readLine().trim(), 10);

        for (let i = 0; i < llCount; i++) {
                const llItem = parseInt(readLine().trim(), 10);
                ll.insertNode(llItem);
        }

        const result = jumpedOn(ll.head)

        console.log(result.join('\n'))
}
```

Evaluation Details

Testcase #8 *(weight:4)*

Status	Passed
Execution time	1.40s

CPU	0s
Memory	29MB
Description	Testcase passed! The solution's output matches the expected output.

Testcase #1 *(sample)*

Status	Passed
Execution time	1.55s
CPU	0s
Memory	28MB
Description	Testcase passed! The solution's output matches the expected output.

Input

```
5
1
2
3
4
5
```

Expected output

```
1
2
4
3
```

Testcase #6 *(weight:4)*

Status	Passed
Execution time	1.74s
CPU	0s
Memory	29MB
Description	Testcase passed! The solution's output matches the expected output.

Testcase #7 *(weight:3)*

Status	Passed
Execution time	1.45s
CPU	0s
Memory	28MB
Description	Testcase passed! The solution's output matches the expected output.

Testcase #4 *(weight:5)*

Status	Passed
Execution time	1.50s
CPU	0s
Memory	29MB
Description	Testcase passed! The solution's output matches the expected output.

Testcase #5 *(weight:5)*

Status	Passed
Execution time	1.46s
CPU	0s
Memory	28MB
Description	Testcase passed! The solution's output matches the expected output.

Testcase #3 *(weight:2)*

Status	Passed
Execution time	1.52s
CPU	0s
Memory	28MB
Description	Testcase passed! The solution's output matches the expected output.

Testcase #2 *(weight:3)*

Status	Passed
Execution time	1.53s
CPU	0s
Memory	28MB
Description	Testcase passed! The solution's output matches the expected output.

Testcase #9 *(weight:2)*

Status	Passed
Execution time	1.49s
CPU	0s

Memory	28MB
Description	Testcase passed! The solution's output matches the expected output.

Problem 7 : Which Oops Concept

MCQ SCORE: **5**

"Operating the tv with the help of a remote to change the channel" is an example of which **OOPs concept**?

Solution

ACCEPTED SCORE: **5.0** / 5

Encapsulation

CANDIDATE'S ANSWER CORRECT ANSWER

Abstraction

Polymorphism

Inheritance

Problem 8 : OOPs: ATM System Minimum Balance Rule

MCQ | SCORE: **20**

You are implementing an ATM system using OOP principles. The system has two main classes: `Account` and `ATM`. Each account has a unique account number and balance and supports deposits and withdrawals. The `ATM` the class can create new accounts, process deposits and withdrawals, check account balances, and transfer money between accounts. The system also ensures that each account maintains a minimum balance of ₹100 after any transaction (if a transaction would bring the account below ₹100, it fails and does not proceed).

Given the following sequence of operations:

- **Create Accounts** with account numbers: 11111, 22222, 33333.
- **Initial Deposits**

1. Deposit ₹1000 into account 11111.
2. Deposit ₹2000 into account 22222.
3. Deposit ₹1500 into account 33333.

- **Transactions**

1. Withdraw ₹500 from account 11111.
2. Withdraw ₹300 from account 22222.
3. Deposit ₹200 into account 33333.

- **Inter-Account Transfer**
- Transfer ₹450 from account 11111 to account 33333.

Question:

1. What will be the final balance of all the accounts after all the transactions and the transfer?
2. Which transaction will fail due to the minimum balance requirement?

Solution

ACCEPTED | SCORE: **20.0** / 20

Final balances: ₹550 (11111), ₹1700 (22222), ₹1750(33333)

The transaction that fails: Withdraw ₹500 from account 11111.

Final balances: ₹1000 (11111), ₹2000 (22222), ₹1500(33333)

The transaction that fails: All

CANDIDATE'S ANSWER | CORRECT ANSWER

Final balances: ₹500 (11111), ₹1700 (22222), ₹1700 (33333)

The transaction that fails: Transfer ₹450 from 11111 to 33333

Final balances: ₹50 (11111), ₹1700 (22222), ₹1750(33333)

The transaction that fails: None

Problem 9 : RDBMS: Data Integrity

MCQ | SCORE: **5**

Consider a database table Orders in an RDBMS, which contains the following columns:

OrderID (primary key), CustomerName, CustomerAddress, ProductID, ProductName, ProductPrice, and OrderDate. You are tasked with **normalizing** this table to reduce **data redundancy** and improve **data integrity**.

After analyzing the table, you decide to apply normalization principles. Which of the following options best describes the initial steps you should take to normalize this table, assuming it is currently in the **First Normal Form** (1NF)?

Solution

ACCEPTED | SCORE: **5.0** / 5

CANDIDATE'S ANSWER	CORRECT ANSWER
Move ProductName and ProductPrice to a new table Products with ProductID as the primary key to achieve Second Normal Form (2NF), since these attributes depend on ProductID, which is part of a composite key.	
Create a new table Customers with CustomerName as the primary key and CustomerAddress as an attribute, and replace CustomerName in the original Orders table with CustomerID to achieve Second Normal Form (2NF) by removing partial dependencies.	
To achieve Third Normal Form (3NF), remove OrderDate from the Orders table and place it in a new table OrderDates, with OrderID as the primary key, because OrderDate is not directly related to the primary key OrderID.	
Split Orders into two tables, one for OrderDetails (OrderID, ProductID, OrderDate) and one for Customer (CustomerName, CustomerAddress), to achieve Third Normal Form (3NF) by eliminating transitive dependencies.	

Problem 10 : HTML DOM

MCQ | SCORE: **5**

How is everything treated in **HTML DOM**?

Solution

ACCEPTED | SCORE: **5.0** / 5

Attributes

Elements

CANDIDATE'S ANSWER | CORRECT ANSWER

Node

All of the above